

Evolution: One-Pager System

ABSTRACT

The original one-pager generator had a critical flaw.

1. Introduction

The original one-pager generator had a critical flaw. It was supposed to create exactly two pages, but it was generating four pages instead. Even worse, it was reporting that it had successfully created two pages when it clearly hadn't. This happened because the system was using a simple rule of thumb: it assumed that between 8,000 and 12,000 characters of HTML would equal two pages. But this assumption was completely wrong. Different content, different fonts, and different layouts all affect how much text fits on a page, so this character-counting method was unreliable.

2. Methodology

The new system starts by selecting a reasonable number of ideas to include. It generates a PDF with that content and counts the actual pages. If it gets exactly two pages, it's done. If it gets more than two pages, it reduces the content by about 25 percent and tries again. If it gets fewer than two pages, it increases the content by about 30 percent. This process continues until it either hits exactly two pages or reaches the maximum number of attempts. The result is a system that actually delivers on its promise of creating two-page documents. This represents something important: the system evolved itself. When the first version failed, we didn't just fix a bug. We created a new version that uses a fundamentally different approach. The new version measures reality instead of estimating it. It adapts based on feedback instead of assuming it got it right the first time. This is the kind of improvement that makes systems more reliable and more capable over time. Now that we have a system that actually works, we can use it to create one-pagers from chat sessions. Each one-pager will be exactly two pages, with clean formatting, and will accurately summarize the key points from a conversation. The system tracks everything it does, so we can see how it evolves over time and learn from what works best.

3. Conclusion

We created an evolved generator that actually measures what it produces. Instead of guessing based on character count, the system generates a PDF, counts the real number of pages using a library called pypdf, and then adjusts the content if needed. It tries up to five times to get exactly two pages. If it generates too many pages, it reduces the amount of content. If it generates too few, it adds more. This creates a feedback loop: the system measures what it created, compares it to the goal, and adjusts until it gets it right. We also discovered that the output had formatting problems. The system was including markdown syntax like hash marks for headers and asterisks for bold text directly in the final PDF. This made the documents look unprofessional. We added a cleaning step that removes all markdown formatting before the content is rendered. Headers lose their hash marks, bold text loses its asterisks, and the output is clean and readable. The original implementation generated four pages but claimed success. The evolved generator generates exactly two pages and accurately reports its success. The original used unreliable estimates. The evolved version uses real measurements. The original couldn't improve because it had no way to know it was wrong. The evolved version can improve because it measures what it actually produces.